CSA Project A: Performance Modelling - RISC-V Processor

Phase 2: Single-Stage Processor and Five-Stage Processor

By:

NEHA PATIL

Np2998

TASKS:

1. Draw the schematic for a single stage processor and fill in your code in the to run the simulator.

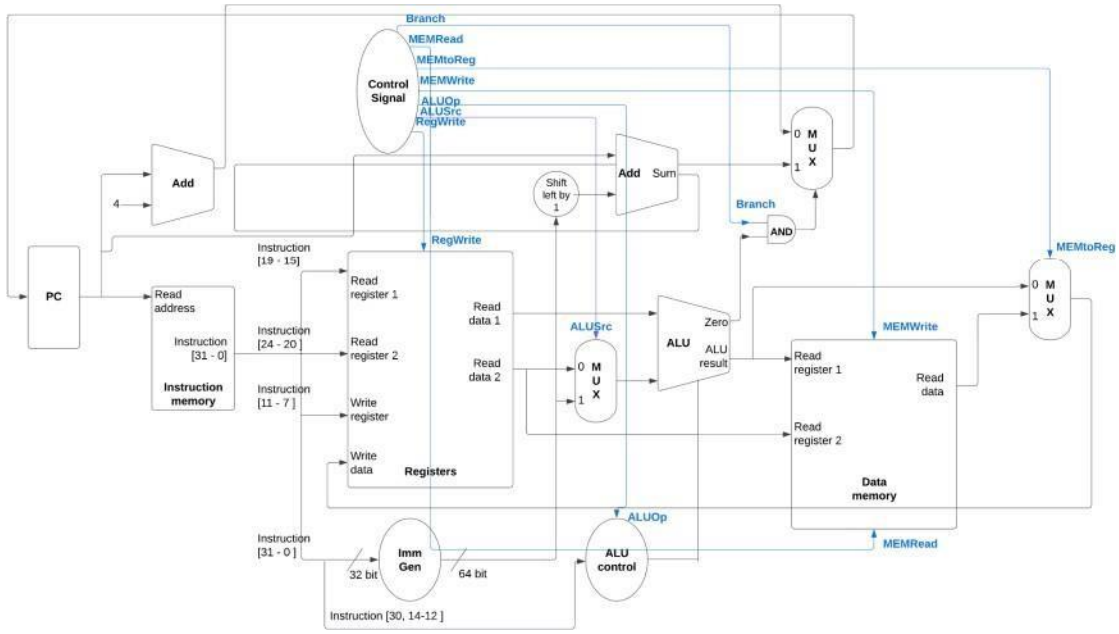
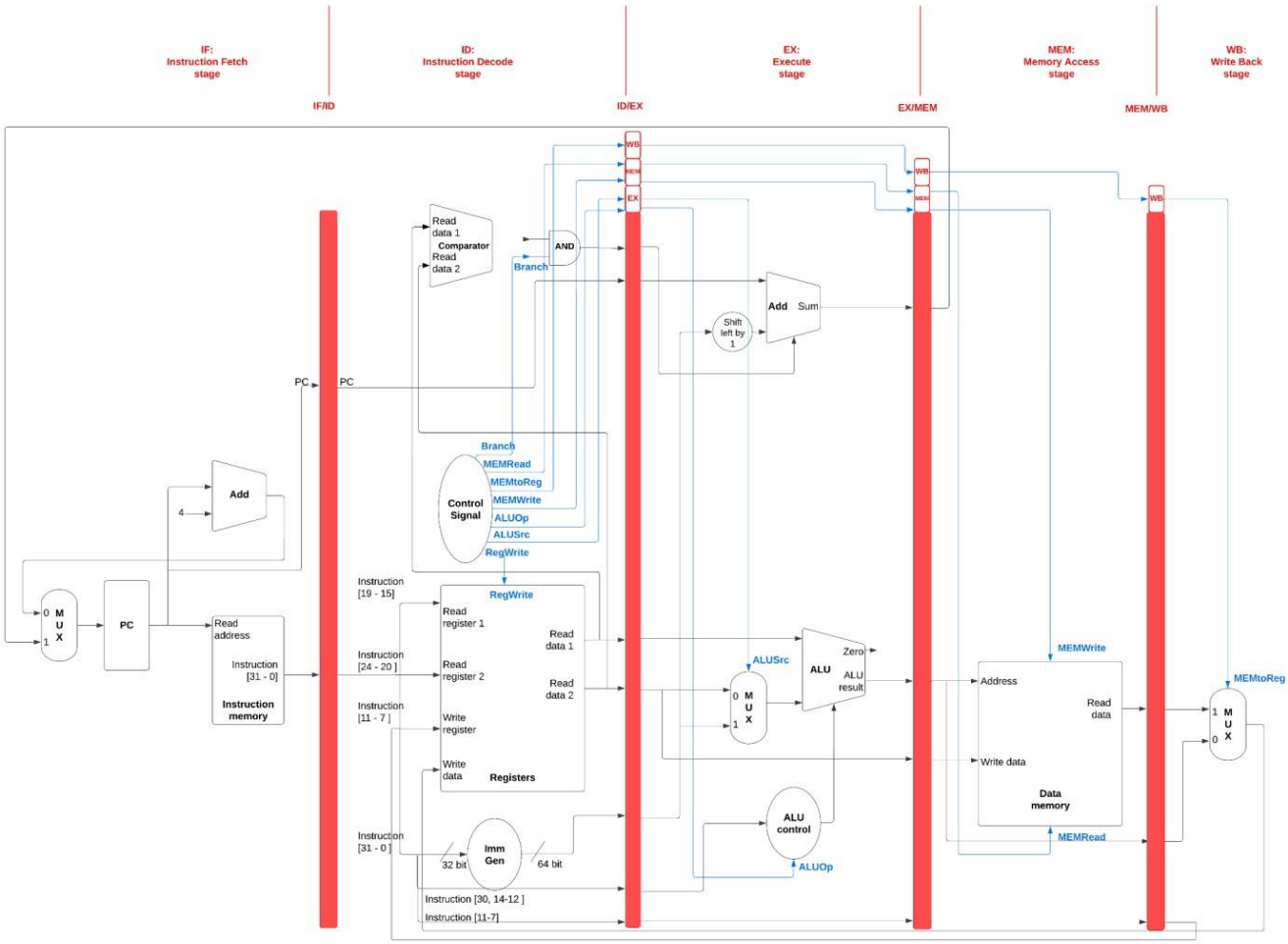


Fig. Single Stage Processor (source. – Computer Organization and Design RISC-V Edition)

The implementation of the single stage processor is given in the class '*SingleStageCore'* in the main.py file

2. Draw the schematic for a five-stage pipelined processor and fill in your code to run the simulator. The processor should be able to take care of RAW and control hazards by stalling and forwarding



The implementation of the five-stage pipelined processor is given in the class *'FiveStageCore'* in the main.py file

3. Measure and report average CPI, Total execution cycles, and Instructions per cycle for both these cores by adding performance monitors to your code.

   i) Testcase 1:

Single Stage Core Performance Metrics:
Number of Cycles taken: 6
Number of instructions taken: 5
Cycles per instruction: 1.200000000000000
Instructions per cycle: 0.833333333333333

Five Stage Core Performance Metrics:
Number of Cycles taken: 10
Number of instructions taken: 5
Cycles per instruction: 2.0
Instructions per cycle: 0.5

```
Single Stage Core Performance Metrics:
Number of Cycles taken:  6
Number of instructions taken:  5
Cycles per instruction:   1.200000000000000
Instructions per cycle:   0.833333333333333

Five Stage Core Performance Metrics:
Number of Cycles taken:  10
Number of instructions taken:  5
Cycles per instruction: 2.0
Instructions per cycle: 0.5
```

ii) Testcase 2:

Single Stage Core Performance Metrics:
Number of Cycles taken: 40
Number of instructions taken: 39
Cycles per instruction: 1.025641025641026
Instructions per cycle: 0.975000000000000

Five Stage Core Performance Metrics:
Number of Cycles taken: 46
Number of instructions taken: 39
Cycles per instruction: 1.1794871794871795
Instructions per cycle: 0.8478260869565217

```
Single Stage Core Performance Metrics:
Number of Cycles taken:  40
Number of instructions taken:  39
Cycles per instruction:   1.025641025641026
Instructions per cycle:   0.975000000000000

Five Stage Core Performance Metrics:
Number of Cycles taken:  46
Number of instructions taken:  39
Cycles per instruction: 1.1794871794871795
Instructions per cycle: 0.8478260869565217
```

iii) Testcase 3:

Single Stage Core Performance Metrics:
Number of Cycles taken: 28
Number of instructions taken: 27
Cycles per instruction: 1.037037037037037
Instructions per cycle: 0.964285714285714

Five Stage Core Performance Metrics:
Number of Cycles taken: 38
Number of instructions taken: 27
Cycles per instruction: 1.4074074074074074
Instructions per cycle: 0.7105263157894737

```
Single Stage Core Performance Metrics:
Number of Cycles taken:  28
Number of instructions taken:  27
Cycles per instruction:   1.037037037037037
Instructions per cycle:   0.964285714285714

Five Stage Core Performance Metrics:
Number of Cycles taken:  38
Number of instructions taken:  27
Cycles per instruction: 1.4074074074074074
Instructions per cycle: 0.7105263157894737
```

4. Compare the results from both the single stage and the five stage pipelined processor implementations and explain why one is better than the other.

Aspect	**Single-Stage Processor**	**Five-Stage Pipelined Processor**
Performance	**Lower throughput:** Each instruction is executed sequentially.	**Higher throughput:** Enables concurrent execution of multiple instructions.
Latency	**Higher latency per instruction:** Execution takes longer due to sequential processing.	**Lower latency per instruction:** Reduced latency through concurrent processing.
Complexity	**Simple design:** Less complexity in architecture.	**Complex design:** Incorporates additional pipeline stages and requires hazard management.
Resource Utilization	**Limited resource utilization:** Idle cycles occur between instructions.	**Improved resource utilization:** Optimizes instruction overlap during execution.
Efficiency	**Lower efficiency:** Sequential processing leads to reduced performance.	**Higher efficiency:** Parallel instruction processing boosts performance
Pipeline Hazards	**No pipeline hazards:** No need to address hazards	**Pipeline hazards present:** Requires managing issues like data and control hazards.
Suitability	**Suited for basic or educational purposes:** Not ideal for high-performance tasks.	**Optimized for modern processors:** Well-suited for high-throughput and efficient computing.

- **Pipelined Processor (Five-Stage):** This design is generally preferred for its superior performance and efficiency. By enabling concurrent instruction execution, it achieves higher throughput and lower latency. While pipelining introduces challenges such as data and control hazards, these issues can be addressed using techniques like forwarding, branch prediction, and hazard detection mechanisms.
- **Single-Stage Processor:** Though simpler in architecture, it has lower performance and efficiency due to sequential instruction execution. This type of processor is typically used in basic or educational settings and is not suitable for modern high-performance computing needs.

Advantages of a five-stage pipelined processor compared to a single-stage processor:

a) **Enhanced clock rate and throughput:** The division of processing into five pipeline stages reduces the amount of logic handled in each stage, allowing for a faster clock rate and improved processor throughput.

b) **Optimized cycle time:**

- In a single-stage processor, the cycle time is determined by the total time needed to process each instruction sequentially.
- For instance, with m instructions each requiring n seconds, the total processing time is $(m \times n)$ seconds.

- In contrast, a five-stage processor distributes the workload across stages, where the cycle time is governed by the slowest stage, approximately ($m \times n / 5$).
- Even with the slowest stage as the determining factor, the pipelined processor achieves significantly faster performance than its single-stage counterpart.

For example, let us consider the testcase 2,

Metric	Single-Stage Processor	Multi-Stage Processor
Execution Time (seconds)	40 * n	46 * n
Approximate Instructions	39	39
Cycles	40	46
IPC (Instructions per Cycle)	0.9750009	0.847824
CPI (Cycles per Instruction)	1.02564	1.17949

5. What optimizations or features can be added to improve performance?

In recent years, single-stage processor designs have fallen out of use due to their inefficiency in handling the complexities of modern computing systems. In single-cycle implementations, control signals often directly manipulate various components of the processor's datapath. When attempting to combine or simplify these control signals, it is crucial to carefully analyze the dependencies between instructions and their associated control signals.

Features to Enhance Processor Performance:

1. **Optimized ALU operation selection:** Replacing a multiplexer with an exclusive OR (XOR) for choosing the ALU operation result can accelerate circuit performance. Additionally, using one-hot encoding within binary encoding for control signal generation improves the ALU's operating frequency.
2. **Pipelined architecture:** Implementing a pipeline allows multiple instructions to be processed concurrently by breaking tasks into smaller stages, avoiding the need to wait for one instruction to complete before starting the next.
3. **Instruction-level parallelism:** By executing multiple independent instructions simultaneously, the processor can maximize resource utilization and overall efficiency.
4. **Multicycle implementation:** This approach reuses functional units across different clock cycles for a single instruction. It allows instructions to take varying numbers of cycles and facilitates resource sharing within a single instruction's execution, offering greater flexibility and efficiency compared to single-cycle designs.
5. **Multi-level cache hierarchies:** Incorporating caches at multiple levels significantly reduces access times by storing frequently used data and instructions closer to the processor, minimizing reliance on slower main memory.